

ALE 課程 8：Inbound 客製化開發與錯誤處理

Lecture

ale07 讓資料流出去，這題處理**接收方**：IDoc 抵達後，誰來處理、處理失敗怎麼辦。Inbound 是 ALE 開發裡最容易出問題、也最需要「可重試」設計的一段——因為發送方已經送完就不管了（ale01 講的廣播式特性），接收方處理失敗的 IDoc 會停在資料庫裡等你處理，這正是 BD87 存在的理由。

Inbound 處理鏈：從 IDoc 抵達到 Function Module 被呼叫

IDoc 抵達（狀態 64：Ready to be passed to application）

- WE20 Inbound Parameters：依 Message Type 找到 Process Code
- WE42 Process Code：指向要呼叫的 Function Module
- （WE57 + BD51）：登記這個 FM 是合法的 Inbound FM 及其處理特性
- Function Module 執行，回填 IDOC_STATUS
- 框架依你回填的狀態更新 IDoc：53（成功）或 51（失敗）

四個設定點各管一件事，缺一不可：**WE20** 的 **Inbound Parameters** 決定「這個訊息類型用哪個 **Process Code**」（ale02 常見誤區 Q&A 提過），**WE42** 決定「這個 **Process Code** 對應哪個 **FM**」，**WE57** 決定「這個 **FM** 可以處理哪個 **Basic Type / Message Type**」，**BD51** 決定「這個 **FM** 的處理特性」（如單筆 / 批次輸入）。

Inbound Function Module 的標準介面（不能自己發明）

框架用固定的介面呼叫你的 FM，2026-09-21 實測讀標準的 **IDOC_INPUT_MATMAS01 / IDOC_INPUT_CLFMAS** 確認：

參數	方向	說明
input_method / mass_processing	IMPORTING	框架傳入的處理模式旗標
workflow_result	EXPORTING	整體結果： 0 = 成功， 99999 = 有錯誤
application_variable / in_update_task / call_transaction_done	EXPORTING	應用層旗標，本題都留空
idoc_contr1	TABLES (LIKE edidc)	這次要處理的 IDoc 控制記錄（可能一次多筆）
idoc_data	TABLES (LIKE edidd)	全部 IDoc 的資料記錄，靠 docnum 對應到各筆
idoc_status	TABLES (LIKE bdidocstat)	你要回填的結果——每處理完一筆 IDoc，就 APPEND 一行： docnum 、 status （ 53 / 51 ）、訊息（ msgty / msgid / msgno / msgv1... ）

參數	方向	說明
<code>return_variables</code>	TABLES (LIKE <code>bdwfretvar</code>)	給 Workflow 用： <code>wf_param</code> 填 <code>Processed_IDOCs</code> 或 <code>Error_IDOCs</code> ， <code>doc_number</code> 填 IDoc 號碼
<code>wrong_function_called</code>	EXCEPTION	收到不該由這個 FM 處理的 IDoc 時拋出

這代表 Inbound FM 的「回傳值」不是 **RETURNING**，而是你填的 `idoc_status` 表：填 53，框架就把 IDoc 標記成功；填 51，框架就把它標成失敗並留在系統裡等你處理。

設計原則：FM 是轉接器，邏輯放 Class

這門課一貫的做法（專案規範：商業邏輯寫在 Class 方法，不塞進 Function Module）——Inbound FM 只負責「拆 `idoc_data`、呼叫 Class、把結果翻譯成 `idoc_status`」。好處：Class 的解析與驗證邏輯可以用 ABAP Unit 直接測試，不需要真的有 IDoc 走過整條管線。

錯誤處理：51 是資料問題，56 是結構問題

- 狀態 51 (Application document not posted)：IDoc 結構正確、成功抵達並進入你的 FM，是你的驗證邏輯（或標準 BAPI）拒絕了這筆資料——例如本題的「公司代碼未登記」。修好資料（補主檔、補設定）之後可以直接重新處理。
- 狀態 56 (IDoc with errors added)：連結構 / 設定層級就有問題（Segment 對不上、找不到 Process Code.....），要先修設定或編輯 IDoc。
- **BD87 (Status Monitor for ALE Messages)**：依 Message Type / 狀態 / 日期篩出 IDoc，選取後 **Process** 就會重新呼叫你的 FM。這要求 Inbound FM 必須是冪等的——同一筆 IDoc 處理第二次不能產生重複資料。本題用 **MODIFY**（而不是 **INSERT**）寫入 Log 表，並且驗證失敗時完全不寫入資料庫，就是為了這個目的。
- ⚠ **Inbound FM 不能 COMMIT WORK**（en06 的教訓在這裡同樣適用）：框架在你的 FM 回傳後統一 commit，並依你回填的狀態決定 IDoc 最終狀態。

學習目標

- 能講出 Inbound 處理鏈四個設定點（**WE20** Inbound / **WE42** / **WE57** / **BD51**）各自負責什麼
- 能依標準介面寫出 Inbound Function Module，並正確回填 `idoc_status`（53 / 51）與 `workflow_result`
- 能把商業邏輯放在 Class、FM 只做轉接，並用 ABAP Unit 測試 Class
- 能分辨 51 與 56 的成因，並用 **BD87** 在修好資料後重新處理失敗的 IDoc
- 理解 Inbound FM 為什麼必須冪等，並能用「失敗時不寫入 + **MODIFY**」實現
- 再次練習 Partner Profile Inbound 設定（這次是自訂訊息類型 **ZALE06**）

事前準備

- ale06 的 **ZALE06** / **ZALE06_BT01** / Segment 已建立釋出且 **WE82** 已指派；ale07 的 Outbound 設定（**WE20** / **BD64**）與 **ZCL_ALE07_PO_OUT** / **ZR_ALE07_SEND** 已完成
- 準備一張 ale07 用過的採購訂單，並用 **SE16N** 查它的公司代碼（**EKKO-BUKRS**），下面稱為「測試公司代碼」

題目需求

Part A：資料表與 Class / FM 開發 (ABAP)

資料表 (套件 \$TMP，欄位一律引用標準 Data Element)

- **ZALE08_CTRL**：允許接收的公司代碼清單。Key：mandt (mandt)、bukrs (bukrs)。沒有其他欄位。
- **ZALE08_POLOG**：接收記錄。Key：mandt、docnum (edi_docnum)、ebeln (ebeln)、ebelp (ebelp)；非 Key：matnr (matnr)、bukrs (bukrs)、lifnr (elifn)、erdat (erdat)、erzet (erzet)。

Class **ZCL_ALE08_PO_IN**

方法	行為
parse	輸入 EDIDD 表，回傳「表頭 + 明細表」結構： Z1ALE06H 對應表頭、每個 Z1ALE06I 對應一筆明細 (matnr18→matnr)
validate	回傳錯誤文字 (空字串 = 通過)，依序檢查：① 沒有表頭 ② 沒有明細 ③ 公司代碼不存在於 T001 ④ 公司代碼未登記在 ZALE08_CTRL ⑤ 供應商不存在於 LFA1 。錯誤文字要短 (不超過 50 字元)——因為它會被放進狀態記錄的 msgv1 欄位
process	parse→validate；有錯誤就直接回傳失敗 (不寫任何資料)；通過就用 MODIFY 寫入 ZALE08_POLOG ，回傳成功

Function Module **Z_IDOC_INPUT_ZALE06** (Function Group **ZFG_ALE08**)：依上表標準介面實作。逐筆處理 **idoc_contr1**；不是 **ZALE06** 就 **RAISE wrong_function_called**；每筆呼叫 **process**，成功填 53、失敗填 51 (訊息用通用訊息 00 / 398，把文字放 **msgv1**)；只要有任一筆失敗，**workflow_result** 就是 99999。

Report **ZR_ALE08_CTRL**：維護 **ZALE08_CTRL** (選取畫面：公司代碼 + 新增 / 刪除單選)，執行後列出目前清單。(這張表沒辦法用 ADT 建 SM30 維護畫面，用小報表代替。)

ABAP Unit：替 **parse** 與 **validate** 中「不碰資料庫」的檢查 (沒表頭、沒明細) 寫測試；含資料庫檢查的部分交給端對端測試。

Part B：Inbound 設定 (GUI，Partner Profile Inbound 再練習)

1. **WE57**：新增一列——**Module **Z_IDOC_INPUT_ZALE06****、**Function type **F****、**Basic type **ZALE06_BT01****、**Message type **ZALE06****、**Direction **2**** (Inbound)
2. **BD51**：新增 **Z_IDOC_INPUT_ZALE06**，輸入類型選「一次處理一筆」(選項編號依你畫面的說明文字為準，預期是 0)
3. **WE42**：新建 Process Code **ZALE08**，選 **Processing with ALE service**，Identification 選 **Function module**，指定 **Z_IDOC_INPUT_ZALE06**
4. **WE20**：Partner **S4HCLNT130** 的 **Inbound Parameters** 新增一列——**Message Type **ZALE06****、**Process Code **ZALE08****、**Processing by Function Module** 選 Trigger Immediately

Inbound/Outbound 判斷提醒：ale07 你在同一個 Partner 的 Outbound 區塊設了 **ZALE06**，這題在 Inbound 區塊又設一次——自我迴圈才會兩個區塊出現在同一個畫面；真實雙系統時，這一步是在接收方系統做的。

Part C：端對端——失敗、修正、重新處理

1. 確認 `ZALE08_CTRL` 是空的。用 `ZR_ALE07_SEND` (取消勾選測試模式) 送出採購訂單。預期：Outbound IDoc 狀態 `03`，Inbound IDoc 停在 `51`，訊息是「公司代碼未登記」
2. 用 `ZR_ALE08_CTRL` 把測試公司代碼加進 `ZALE08_CTRL`
3. `BD87`：Message Type `ZALE06`、狀態 `51`，選取該 IDoc → **Process**。預期：狀態變成 `53`，`ZALE08_POLOG` 出現對應資料列
4. 再送一次同一張採購訂單，確認新 IDoc 直接走到 `53` (因為設定已經就緒)
5. 反向測試：`ZR_ALE08_CTRL` 刪除該公司代碼，再送一次，確認又回到 `51`——證明驗證邏輯是真的在擋，不是之前剛好通過

參考答案 (驗證方式)

參考答案程式碼已快照在本目錄：`zale08_ctrl.tabl.abap`、`zale08_polog.tabl.abap`、`zcl_ale08_po_in.clas.abap`、`zcl_ale08_po_in.clas.testclasses.abap`、`z_idoc_input_zale06.func.abap`、`zr_ale08_ctrl.prog.abap`。

⚠ 狀態：草稿，尚未經 **SAP** 語法檢查與執行驗證 (同 `ale07`，依賴 `ale06` 的 Segment 才能編譯)。待驗證清單見 README。

驗證要點 (完成後回報，我會核對)：

- `SELECT DOCNUM, DIRECT, MESTYP, STATUS FROM EDIDC WHERE MESTYP = 'ZALE06' ORDER BY DOCNUM DESCENDING`——Outbound (`DIRECT = 1`) 與 Inbound (`DIRECT = 2`) 各一筆
- `SELECT DOCNUM, COUNTR, STATUS, STATXT FROM EDIDS WHERE DOCNUM = '<Inbound IDoc號>' ORDER BY COUNTR`——完整歷程：`64`→`51`→(`BD87` 後) `53`
- `SELECT * FROM ZALE08_POLOG`——確認資料列、`docnum` 對應到 Inbound IDoc
- `SELECT MESTYP, SNDPRN, METHOD FROM EDP21 WHERE MESTYP = 'ZALE06'`——`METHOD` 欄位即 Process Code (`ZALE08`)

思考題

1. 為什麼驗證失敗時要「完全不寫入資料庫」？如果先寫入一半再回報 `51`，`BD87` 重新處理時會發生什麼事？
2. Log 表的 Key 包含 `docnum`。如果發送方因為網路問題重送了「同一張採購訂單的另一筆新 IDoc」，接收方會有兩批 Log。你覺得這算問題嗎？如果要避免重複處理，要用什麼當「業務冪等鍵」？(提示：想想 `ebeln`，以及「同一張單據更新過」跟「單純重複」如何區分)
3. 標準 FM `IDOC_INPUT_MATMAS01` 有 `mass_processing` 參數，你的 FM 完全沒用到。什麼情況下你必須認真處理這個參數？(提示：想想「一次收到 1000 筆 IDoc」時，逐筆 commit 與批次處理的效能差異)

答案

Part A 的參考答案程式碼見上方快照檔；Part B 為 GUI 設定 (`WE57` / `BD51` / `WE42` / `WE20` 皆無 ADT API)；Part C 的端對端結果請回報各步驟的 IDoc 號碼與狀態碼。